

schengen: Constraint-Guided Schematic Generation for Analog Circuits via Large Language Models

Anonymous Authors
Anonymous Institution(s)

Abstract—Schematic generation is essential in analog synthesis and topology design, enabling design understanding, expert feedback, and documentation. Recent approaches based on reinforcement learning (RL) and large language models (LLMs) formulate this task as optimization or translation from SPICE netlists to schematics. However, they are often computationally expensive, limited to small circuits, or lack controllability. Constraint-driven placement is long established in analog automation, but its constraints come from expert knowledge or automated subcircuit recognition, both unavailable precisely on the novel and synthesized topologies that recognizers do not cover. We introduce *schengen*, which applies constraint-driven placement to schematic generation and adds a large language model as a third constraint source for exactly that regime, with an interactive GUI for fine-grained control. Our contribution is to quantify what this source adds. Across 15 benchmark circuits (14–48 devices) and 10 random seeds each, LLM-generated constraints reduce bends, net crossings, and HPWL by 21%, 27%, and 17% over a classical constraint-driven configuration. The effect is reproduced by two independent models from an identical prompt. The gain does not come from a weakened baseline: it is *larger* (36% fewer crossings) on the circuits where subcircuit recognition is available and the classical configuration is at full strength, while block alignment itself is unchanged. The LLM improves what the recognizers do not reach. All source code, benchmarks, and generated schematics are available at: <https://anonymous.4open.science/r/schengen>.

Index Terms—Schematic generation, Large language models, Analog circuits

I. INTRODUCTION

Analog topology discovery and synthesis [1], [2], [3] have advanced rapidly with automation and machine learning, enabling large-scale generation of SPICE netlists. However, these netlists are low-level and difficult to interpret, limiting their usefulness for design analysis, debugging, and expert feedback. Converting SPICE netlists into human-readable schematics is therefore a critical step for design understanding, validation, and documentation. Recent netlist-to-schematic approaches include LLM-based methods [4], RL [5], [6], and conventional optimization [7]. While promising, they suffer from high computational cost (e.g., RL training), limited scalability beyond small circuits, and a lack of fine-grained controllability that would let designers guide or refine results. To address these limitations, we adopt the *constraint-driven* methodology that is long established in analog layout automation [8], [9] and apply it to schematic generation. Instead of generating schematics directly, we define a set of constraint primitives that govern component placement and routing on a grid-based canvas. The resulting constraint system is solved using a SAT/CP (i.e., constraint programming) solver integrated with a custom placement and routing engine. Constraints in that methodology conventionally come from

expert knowledge or from automated subcircuit recognition, and both are unavailable precisely where they are most needed: on novel or synthesized topologies that recognizers do not cover. Our central claim is that a large language model is an effective third source, supplying structural placement constraints for circuits where the classical sources are silent. Based on this formulation, we develop *schengen*, an automated tool for schematic generation from SPICE netlists using constraint programming, which also supports interactive refinement so that designers can improve readability and structural quality. Our main contributions are as follows:

- We bring the constraint-driven methodology of analog layout automation to schematic generation, and identify the LLM as a constraint source that covers the topologies rule-based recognizers miss.
- We quantify what that source adds over classical constraint-driven placement: across 15 circuits and 10 seeds, LLM-generated constraints reduce crossings by 27%, bends by 21% and HPWL by 17%, and halve seed-to-seed variability. The gain is reproduced by two independent models from an identical prompt, and does not depend on a weakened baseline: it is *larger* (36% fewer crossings) on the circuits where subcircuit recognition is available and the classical baseline is at full strength.
- We present *schengen*, an automated framework combining constraint solving with placement and routing, offering an interactive, human-in-the-loop workflow that improves interpretability, controllability, and schematic quality.

II. RELATED WORK

Two largely separate lines of work bear on this problem. The first is *schematic generation* itself: early heuristic optimization and staged placement for digital circuits [10], [11], [12], [13], reinforcement learning for analog designs [5] and PCB reverse engineering [6], and recent LLM-based approaches such as Schemato [4] for netlist-to-LTSpice conversion, frameworks generating SKiDL code and PCB layouts [14], [15], and PCBSchemaGen [16] using knowledge graphs. These target a readable diagram, but none of them is constraint-driven. The second is *constraint-driven analog automation*, which is mature and long predates this work: it integrates expert knowledge into the design flow [8] and derives structural constraints automatically, by rule-based extraction [17] or machine learning [9], [18], [19], to guide topology and layout. EDA companies have patented device clustering and rule-based group placement on the same principle [20], [21], [22].

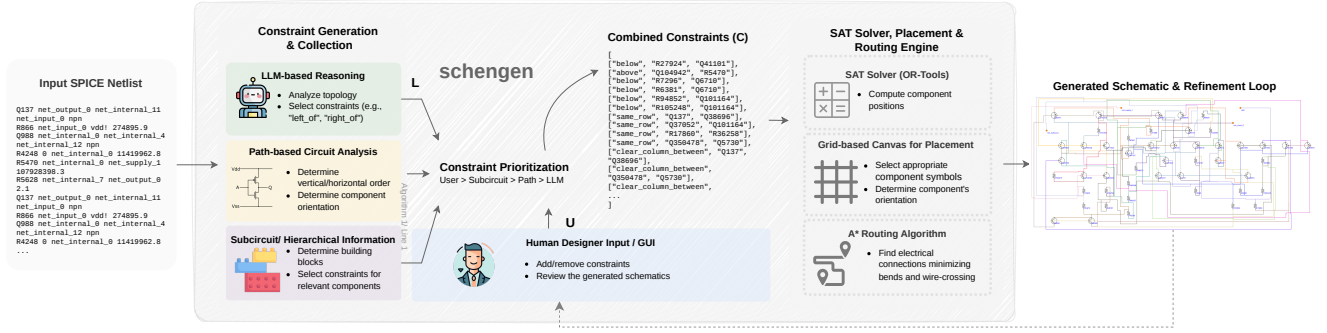


Fig. 1. Overview of the schengen framework (Sec. III).

We claim no novelty for the constraint formulation itself. Its target, however, is physical layout, and commercial tools still fail to produce aesthetic *schematics* for many analog circuits. *schengen* sits at the intersection: it brings the constraint-driven methodology to schematic generation, and identifies the LLM as a constraint source that covers the topologies rule-based recognizers miss. That gap is not hypothetical: automated subcircuit extraction yields no groups at all for 10 of our 15 benchmark circuits, exactly the regime in which the extraction lineage has nothing to contribute. Unlike RL methods requiring extensive training [5], [6] or direct LLM generation that struggles on large circuits [4], the resulting system is training-free and produces valid, connected schematics by construction. What we evaluate is therefore not whether constraints help, which is established, but how much an LLM adds on top of classical constraint sources.

III. THE PROPOSED METHOD

A. Method Overview

Figure 1 illustrates the proposed *schengen* framework, which supports common analog components including MOSFET/BJT transistors, capacitors, and resistors. To reduce the continuous design space, we adopt a grid-based canvas, where each cell is indexed from (0,0) at the bottom-left. All components are initially unassigned to grid locations. Placement is formulated as a constraint generation problem, where constraints derived from LLM reasoning, heuristic rules (e.g., path-based analysis), and user input define relative positions and orientations (e.g., alignment, adjacency). Constraint primitives are detailed in Sec. III-B. The full placement procedure is given as pseudocode in suppl. C. The combined constraint set C aggregates constraints from circuit analysis and subcircuit recognition, from user input, and from iteratively admitted LLM-generated constraints. A constraint programming solver (e.g., OR-Tools [23]) then computes feasible placements S satisfying C while optimizing three routing-aware objectives: compactness (maximum occupied rows), Half-Perimeter Wire Length (HPWL), and estimated routing cost (total Manhattan distance). Finally, given the placement, an A*-based routing algorithm establishes electrical connections, producing the complete schematic.

B. Constraint Set Primitives

schengen provides 16 constraint primitives, all following the uniform format [$\langle \text{constraint} \rangle$, ($\langle \text{value} \rangle$), $\langle \text{target} \rangle$, $\langle \text{references} \rangle$], where $\langle \text{constraint} \rangle$ is the spatial or structural relation, $\langle \text{target} \rangle$ the component constrained, $\langle \text{references} \rangle$ the components it depends on, and the optional $\langle \text{value} \rangle$ an additional parameter. For example, [left_of , $m1$, $m2$] enforces that the column index of $m1$ is smaller than that of $m2$. Others include *same_row*, *same_column*, *cluster_proximity* and *t_junction*. The complete list is given in suppl. A, and each is translated into solver-compatible form (e.g., OR-Tools constraints).

schengen additionally enforces implicit default constraints that capture widely adopted design conventions: **(D1)** preventing component overlap, **(D2)** enforcing minimum spacing between components (e.g., at least two rows and columns apart), **(D3)** avoiding placement near the canvas boundaries, and **(D4)** positioning VDD terminals at the top and GND terminals at the bottom (if present). These are applied by default without over-constraining the solution space. No predefined constraints are imposed on input/output terminals. Their placement is determined by the constraint sources described below.

C. Constraint Generation with LLMs

Previous work [24], [25] shows that LLMs can analyze SPICE netlists and identify functional building blocks, so we use them to generate the placement constraint set. The input is the SPICE netlist and the constraint primitives, optionally with free-form subcircuit or partitioning descriptions to strengthen the model’s reasoning. The model then selects constraints from the circuit structure and functionality. Coverage is the reason this source matters: rule-based recognizers lack heuristics for unseen topologies [3] and learning-based ones lack training data [19], [24], whereas the model needs neither.

A simplified prompt is shown in suppl. F. Two of its sections are substituted automatically rather than written by hand (the circuit’s SPICE netlist, and the vocabulary of constraint primitives, rendered from the same registry the solver uses), so the prompt cannot offer the model a primitive the solver does not implement, and cannot be tuned circuit by circuit. Every circuit and both models in Sec. V receive the identical template, so the comparison in Table I isolates the model and

not the prompt. The resulting constraints complement those from the other sources described below. They are incorporated iteratively, each admitted only if it does not conflict with higher-priority constraints, so that LLM suggestions enhance rather than degrade the placement. The pseudocode specifies *which* constraints are admitted. The implementation realizes the same admitted set more efficiently (suppl. C). We further prompt the LLM to follow an *iterative strictness* strategy: begin with constraints that preserve placement flexibility (e.g., `right_of`) and progressively introduce stricter ones that sharply reduce the feasible space (e.g., `same_row`, which enforces exact alignment). This supports an interactive workflow of successive refinement.

D. Enhancing Schematic Quality with Additional Constraint Sources

`schengen` aggregates constraints from two further sources, which reduce the solution space and improve consistency.

1) *Subcircuit and Partitioning Information*: Subcircuit and partitioning information provide strong structural priors for placement, obtainable via heuristics, template matching, or ML-based methods. For operational amplifiers we use ACST [17], [26] for accurate rule-based extraction. For general circuits, ML approaches such as GANA [19] and GENIE-ASI [24] can be used, although false positives may degrade placement quality. `schengen` maintains a library of common subcircuits (e.g., differential pairs, current mirrors, inverters) with predefined placement constraints that encode domain knowledge such as alignment and symmetry (suppl. F). Once a subcircuit is identified, its constraints are applied automatically, ensuring that recognized structures are placed in a human-readable manner.

2) *Path-based Circuit Analysis (Traceability Principles)*: We further derive placement constraints from current flow paths (VDD to GND) and common schematic conventions [5], since components sharing electrical connections are typically aligned in human-designed schematics: **P1**. Same-type components with drain-source connections are placed in the same column, **P2**. Different-type components with drain-drain or source-source connections are placed close in the same column, **P3**. No components are placed between connected components (P1, P2), **P4**. Same-type components sharing gate connections are aligned in the same row, **P5**. Components connected to VDD (GND) are placed in the upper (lower) region, **P6**. Input/output ports are placed near connected components. These constraints improve traceability and readability. All are enabled by default, but users can selectively apply them via the GUI.

3) *User-defined Constraints*: A web-based GUI lets designers define, modify, and remove constraints interactively, in the same format as LLM-generated ones, with immediate visualization.

E. Constraint Prioritization

When multiple constraints are imposed, conflicts may arise. `schengen` therefore applies constraints in the high-to-low priority order: (1) user-defined, (2) subcircuit-based, (3) path-based, and (4) LLM-generated. Higher-priority constraints are enforced first and cannot be overwritten. LLM-generated constraints are incorporated only if they do not violate them. This

preserves critical structural knowledge while allowing LLMs to resolve placement ambiguities, which is especially useful when subcircuit recognition fails, as in novel synthesized topologies. Conflicts are resolved in two stages, and it is worth being precise about which constraint yields. LLM constraints are first admitted against the user- and path-based base: the batch is tested for joint feasibility and, when that fails, bisected so that only the offending constraints are dropped. Each recognized subcircuit group is then tested against the admitted set. A compatible group is accepted outright. A conflicting one is resolved in favour of *the recognizer*, which is kept while the LLM relations it displaces are dropped. A group is discarded only when it cannot be placed against the user- and path-based base at all, the signature of a genuinely mis-recognized group rather than merely an inconvenient one. The ordering is deliberate, and one benchmark circuit shows its cost plainly: on example-4 every recognized group is placeable, so all are kept and the whole LLM set is displaced, leaving that circuit's LLM configurations identical to the baseline by construction. For such cases `schengen` supports a soft-admission variant in which LLM constraints enter the objective as weighted penalties, so the solver selects a maximum-weight satisfiable subset jointly with the placement instead of committing to a drop/keep decision in advance (suppl. D). Hard admission is the default and is used for all results reported here.

F. Determination of Component Orientation

Each component has two orientations: transistors have the gate/base facing right (0) or left (1), and two-terminal devices have the positive terminal up (0) or down (1). Recognized subcircuits define orientations directly (e.g., symmetric devices in a current mirror are assigned opposite orientations, suppl. F). Remaining components face the side carrying more connections, and each transistor not fixed by a subcircuit is finally re-oriented so its gate faces the side on which its gate net lies, which shortens the gate rails that dominate wiring. User-defined orientations override all automatic assignments.

IV. WIRE ROUTING WITH A* SHORT-PATH ALGORITHM

After determining component placement and orientation, `schengen` routes all nets while preserving readability and efficiency. For each net, connected components are identified from the SPICE netlist and pin positions are inferred from component locations, orientations, and symbol templates. Since each component occupies a single grid cell, pins lie at the midpoints of cell edges, making routing unambiguous. Interconnections are then generated using A* search, minimizing wire length and bends. Unlike physical layout, wire crossings are unavoidable in schematics [6], so `schengen` permits controlled crossings under a small penalty, which often yields more compact and visually coherent layouts.

Routing order significantly affects quality, so nets with more connections are routed first, allowing complex nets to establish primary routes early. Routing proceeds incrementally: starting from one pin, each new pin connects to the nearest routed node, forming a trunk-and-branch (Steiner-like) structure, with the A* cost combining Manhattan distance with a congestion penalty

from existing routes. Two refinements target the cases where this scheme reads poorly. Human schematics draw gate-to-gate wires straight through aligned devices, so narrow regions between aligned components are marked routable during search. Supply and bias nets, which connect a large fraction of the devices, would otherwise weave raggedly between them. For such high-fanout nets `schengen` instead commits to a single straight trunk spanning the net’s extent and connects each pin by a short perpendicular stub. This reproduces the power-rail convention of hand-drawn schematics and, because the trunk precedes the stubs, removes the dependence on pin visiting order that makes incremental routing of high-fanout nets unstable.

Finally, a guarded cleanup pass deduplicates coincident segments, removes tails that terminate at no pin, and merges collinear segments to reduce bends. It never removes a segment that is the sole connection to a pin, so it cannot disconnect a routed net. `schengen` then applies the drawing conventions analog designers expect: crossings between unrelated nets are drawn as hops so that a crossing is visually distinguishable from a junction, supply nets are drawn at the top and ground at the bottom, and gate rails are drawn as shared horizontal wires rather than per-device stubs. These conventions do not change placement or connectivity, but they are what the human-readability criterion of Sec. V ultimately measures.

V. IMPLEMENTATION & EVALUATION

A. Tool Implementation

`schengen` is implemented in Python, using OR-Tools [23] as the constraint solver and Flask [27] for the web application. The implementation is publicly available at [28], and the GUI, which loads netlists and settings and adds, removes or selects constraints interactively, is shown in suppl. A. It takes a flat SPICE netlist plus optional subcircuit information, LLM-generated constraints, and user-defined constraints. On an AMD Ryzen Threadripper PRO 5955WX (16 cores, 126 GB RAM, Ubuntu 20.04), the median end-to-end runtime over the 15 benchmark circuits is 14.1s without LLM-generated constraints, 20.6s with Opus 4.8 constraints and 34.0s with the more numerous GPT-5.4 constraints. The configurations scale very differently, however, and that bounds where the method applies. Without LLM constraints runtime grows mildly across the benchmark range, 4.8s at 14 devices to 26.5s at 48. With them it grows steeply, 5.7s to 105.9s for Opus 4.8 and up to 122.8s for the more verbose GPT-5.4. The gap is entirely constraint admission, a median 71% of runtime and scaling with the number of proposed constraints, while placement and routing together stay at a few seconds even on the largest circuit (suppl. C). Admission is thus the one bottleneck between `schengen` and industrial-scale circuits, and also the most tractable, because its feasibility checks are independent but are currently run one after another. Constraint generation is negligible by comparison: one LLM call per circuit, roughly 68k input and 6–11k output tokens for the *entire* benchmark, with no training of any kind. Generated schematics can be exported in `.png` and `.asc` (LTSpice) formats, and rendered from the command line for batch generation.

B. Evaluation

Benchmark Circuits: We separate qualitative from quantitative evaluation. For *qualitative* comparison we use the three circuits having an external reference schematic: a differential operational amplifier [29], a benchmark from [5] for which both a prior method’s and a commercial tool’s schematics are published, and a circuit from our in-house synthesis engine. For the first two, netlists are derived from the reference schematics and subcircuits extracted using ACST [26]. For *quantitative* evaluation we use 15 circuits (14–48 devices): the three above and twelve further operational amplifiers spanning single-, two- and three-stage topologies, single-ended and fully-differential, including nested-Miller-compensated (NMC) and reversed-NMC variants. Subcircuit information is available for only 5 of the 15, reflecting the practical reality that recognition does not cover novel or synthesized topologies. No user-defined constraints are applied to any circuit, and the two LLM configurations use the prompt template of Sec. III-C unchanged, differing only in which model is queried.

Evaluation Metrics: Schemato [4] treats generation as translation and evaluates via *Graph Edit Distance* and *Compilation Success Rate*. `schengen` formulates it as placement and routing, ensuring structural and functional correctness by construction, which makes such metrics vacuous here. We therefore adopt the aesthetic metrics introduced for this purpose by AEM-PCB [6] (wire bends, net crossings, HPWL, and alignment neatness) rather than defining our own. The first three are quantitative, while alignment neatness is, in `schengen`, largely enforced wherever subcircuit templates are available. Given the lack of standardized metrics and limited access to baselines [5], we first perform *qualitative visual comparisons* against reference schematics, assessing (i) visibility of functional blocks and (ii) compactness of placement, and then evaluate routing quality quantitatively across configurations within `schengen`. We state plainly what these metrics are and are not: they are structural proxies for readability, adopted from prior work but not validated here against human judgement, and this paper reports no study in which engineers rate the resulting schematics. Claims below concern the proxies. Whether a designer finds the result easier to read is a question these numbers narrow but do not settle. All schematics are publicly available at [28].

1) Example 1: Standard differential operational amplifier circuit: We compare `schengen` with and without LLM constraints against Schemato, using Opus 4.8, the stronger of the two generators in Table I. As shown in Fig. 2b, Schemato fails to generate a valid schematic: the produced `.asc` file is syntactically incorrect, and after minimal correction using GPT-5.4 it becomes loadable in LTSpice but remains unstructured, with misplaced components and disconnected wires. We report this outcome without attributing it to a cause: the circuit lies outside the size range Schemato was trained on, and we did not test it in-range, so the comparison establishes that constraint-guided placement yields a valid schematic here and not that it produces better schematics under matched conditions. `schengen` (Fig. 2d, 2e) exposes functional blocks:

TABLE I
EFFECT OF LLM-GENERATED PLACEMENT CONSTRAINTS ON ROUTING QUALITY (15 CIRCUITS, 10 SEEDS, LOWER IS BETTER).

Configuration	Routing quality				Reprod.	Cost	
	#crossings	#bends	HPWL	Routing cost (10^3)	σ_{seed}	Completed	Runtime
w/o LLM constraints	661 ± 44	802 ± 36	3322 ± 121	500 ± 23	8.3	149/150	14.1 s
w/ LLM (Opus 4.8)	486 ± 30 (-27%)	630 ± 27 (-21%)	2772 ± 49 (-17%)	343 ± 12 (-31%)	4.3	150/150	20.6 s
w/ LLM (GPT-5.4)	521 ± 34 (-21%)	664 ± 35 (-17%)	2948 ± 72 (-11%)	360 ± 15 (-28%)	5.6	150/150	34.0 s

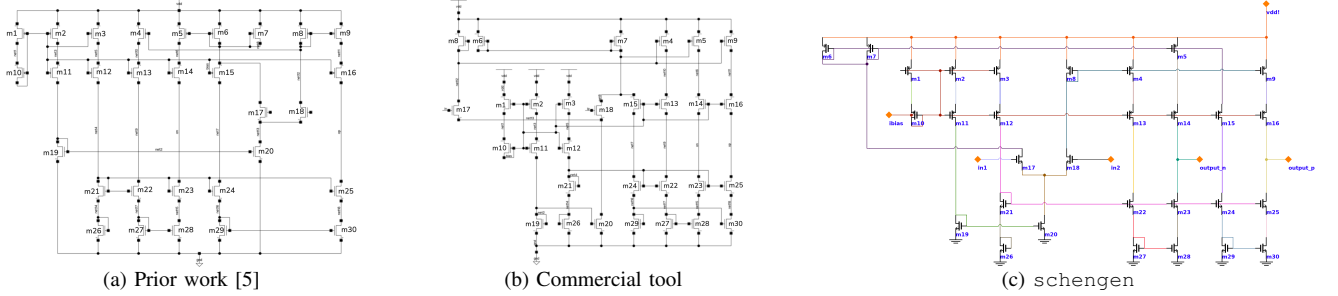


Fig. 3. (Example 2) Schematic comparison: prior work [5], a commercial tool, and schengen.

1166 constraints against 659 for the same circuits, of which a smaller fraction survives admission (75% versus 80%), so it takes $1.7\times$ as long for slightly worse schematics (suppl. E, per circuit). What matters is therefore not how many constraints are proposed but how many survive without over-determining the placement: LLM constraints complement the path- and subcircuit-based sources rather than replacing them. Renders of all 15 circuits are in suppl. F.

VI. DISCUSSION

A. Strength of the Classical Baseline

The classical configuration recognizes subcircuits in only 5 of the 15 circuits, which invites the objection that LLM constraints win only because the baseline is missing its structural priors on the other ten. The data say the opposite. Opus 4.8 reduces crossings by 36% on the five circuits with full ACST recognition and by 24% on the ten with none, both taken from the same runs as Table I and listed per circuit in suppl. E. The gain is *larger* where the baseline is strongest, and that holds even though the first group contains the one circuit whose baseline is identical by construction (Sec. III).

B. Severity of Constraint Conflicts

Admission is stable rather than fragile (per-circuit counts in suppl. E): the median circuit retains 86% of proposed constraints for Opus 4.8 and 81% for GPT-5.4, and apart from example-4, whose LLM set is displaced entirely, the lowest retention is 75% and 48% respectively. Severe conflict is therefore not an LLM-versus-LLM phenomenon: it occurs in one circuit, and there through subcircuit de-confliction.

C. Scope of the Improvement

The improvement is in routing quality, not block structure. Scoring recognized subcircuit groups for alignment and compactness, all three configurations align an identical 46 of 48 groups, and group bounding boxes differ inconsistently (Opus 4.8 1.8% looser, GPT-5.4 8.2% tighter). Where a recognizer supplies

groups it already fixes their layout. The LLM's contribution is to everything the recognizer does not cover.

D. Seed-to-Seed Reproducibility

Beyond routing quality, LLM-generated constraints change how much a single run can be trusted. Because the solver chooses arbitrarily among the equally good placements that the constraint set leaves undetermined, an unconstrained problem yields visibly different schematics on different seeds. The σ_{seed} column of Table I measures that freedom: for each circuit we take the standard deviation of its crossing count over the 10 seeds, and σ_{seed} is the mean of those 15 values. It is a spread, not a level, so a circuit that averages 213 crossings but moves by 22 across seeds contributes 22, not 213. LLM-generated constraints shrink it: the crossing count varies by 8.3 without them and by 4.3 with Opus 4.8 constraints, and HPWL by 24.9 against 7.0, and four of the 15 circuits become fully deterministic across all ten seeds, against two without. This matters for a tool used iteratively: a designer who adjusts one constraint and regenerates should see a schematic that differs because of the edit, not because of an arbitrary solver choice. Reliability does not distinguish the configurations: all three complete essentially every run.

VII. CONCLUSION AND OUTLOOK

We presented *schengen*, which applies constraint-driven placement to schematic generation and adds the LLM as a constraint source for the topologies rule-based recognizers do not cover. Across 15 circuits and 10 seeds it reduces crossings by 27%, bends by 21% and HPWL by 17% over a classical constraint-driven configuration. Three limitations bound these claims, and our future work. The metrics are structural proxies adopted from prior work, not validated here against human judgement. The benchmark spans 14–48 devices: constraint admission dominates runtime and must be parallelized before industrial-scale circuits come into reach.

REFERENCES

- [1] H. Zhang, S. Sun, Y. Lin, R. Wang, and J. Bian, "Analogxpert: Automating analog topology synthesis by incorporating circuit design expertise into large language models," in *2025 International Symposium of Electronics Design Automation (ISED)*, IEEE, 2025, pp. 772–777.
- [2] Y. Hou et al., "Cktgen: Specification-conditioned analog circuit generation," *arXiv e-prints*, arXiv:2410.2024.
- [3] I. Abel and H. Graeb, "Fuboco: Structure synthesis of basic op-amps by functional block composition," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 27, no. 6, pp. 1–27, 2022.
- [4] R. Matsuo et al., "Schemato—an llm for netlist-to-schematic conversion," in *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*, IEEE, 2025, pp. 1–7.
- [5] H.-Y. Hsu and M. P.-H. Lin, "Automatic analog schematic diagram generation based on building block classification and reinforcement learning," in *Proceedings of the 2022 ACM/IEEE Workshop on Machine Learning for CAD*, 2022, pp. 43–48.
- [6] J. Yang, K. Qiao, S. Shi, B. Song, J. Chen, and B. Yan, "Aem-pcb reverser: Circuit schematic generation in pcb reverse engineering using reinforcement learning based on aesthetic evaluation metric," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1608–1612, 2023.
- [7] B. G. Arsintescu, "A method for analog circuits visualization," in *Proceedings international conference on computer design. VLSI in computers and processors*, IEEE, 1996, pp. 454–459.
- [8] G. Jerke and J. Lienig, "Constraint-driven design: The next step towards analog design automation," in *Proceedings of the 2009 international symposium on Physical design*, 2009, pp. 75–82.
- [9] O. Mitea, M. Meissner, L. Hedrich, and P. Jores, "Automated constraint-driven topology synthesis for analog circuits," in *2011 Design, Automation & Test in Europe*, IEEE, 2011, pp. 1–4.
- [10] B. Naveen and K. Raghunathan, "An automatic netlist-to-schematic generator," *IEEE Design & Test of Computers*, vol. 10, no. 1, pp. 36–41, 2002.
- [11] S. T. Frezza and S. P. Levitan, "Spar: A schematic place and route system," *IEEE transactions on computer-aided design of integrated circuits and systems*, vol. 12, no. 7, pp. 956–973, 1993.
- [12] N.-H. Kim, K.-S. Kim, K.-M. Choi, and J.-T. Kong, "Righttopologizer: An efficient schematic generator for multi-level optimization," in *Proceedings of 13th Annual IEEE International ASIC/SOC Conference (Cat. No. 00TH8541)*, IEEE, 2000, pp. 387–391.
- [13] B. Garg, A. Singh, and A. Agrawal, "Efficient undo-redo with incremental netlist to schematic generation," in *2010 53rd IEEE International Midwest Symposium on Circuits and Systems*, IEEE, 2010, pp. 789–792.
- [14] K. S. A. Hasan, S. R. Raiyan, H. M. Alvee, and W. Sadik, "Circuitlm: A multi-agent llm-aided design framework for generating circuit schematics from natural language prompts," *arXiv preprint arXiv:2601.04505*, 2026.
- [15] H. Du, X. Du, and J. Yang, "Asgf4epd: An automated schematic generation framework for electronic products design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2026.
- [16] H. Zou, P. Han, E. Nazerian, and A. Q. Huang, "Pcbschemagen: Constraint-guided schematic design via llm for printed circuit boards (pcb)," *arXiv preprint arXiv:2602.00510*, 2026.
- [17] I. Abel, M. Neuner, and H. Graeb, "A functional block decomposition method for automatic op-amp design," *Integration*, vol. 85, pp. 108–120, 2022.
- [18] P. Mangalagiri et al., "Cdls: Constraint driven generative ai framework for analog layout synthesis," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [19] K. Kunal et al., "Gana: Graph convolutional network based automated netlist annotation for analog circuits," in *DATE*, 2020, pp. 55–60.
- [20] B. Singh, D. O'Riordan, B. G. Arsintescu, A. Goel, and D. R. Deshpande, *System and method for circuit schematic generation*, US Patent 7,917,877, 2011, 2011.
- [21] A. S. Bikram Garg Rajeev Sehgal, *Schematic generation visualization aid for netlists comprising analog circuits*, US Patent 8,504,953, 2013, 2013.
- [22] T.-H. Tsai, P.-H. Lin, S.-C. Lin, and H.-C. Yu, *Rule-based schematic diagram generator*, US Patent 7,386,823, Jun. 2008.
- [23] Google, *OR-Tools*, 2026. Accessed: Jan. 30, 2026. [Online]. Available: <https://developers.google.com/optimization>
- [24] P. Pham et al., "Genie-asi: Generative instruction and executable code for analog subcircuit identification," in *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*, IEEE, 2025, pp. 1–21.
- [25] A. Dăescu, A. Guzu, G. Nicolae, and C. Dan, "Evaluating the impact of circuit representation on llm-based functional block recognition in analogue circuits," *Electronics Letters*, vol. 62, no. 1, e70512, 2026.
- [26] ACST: Analog Circuit Synthesis Tool, 2026. Accessed: Jan. 30, 2026. [Online]. Available: <https://github.com/inga000/acst>
- [27] Pallets, *Flask Documentation (3.1.x)*, 2026. Accessed: Apr. 6, 2026. [Online]. Available: <https://flask.palletsprojects.com/en/stable/>
- [28] Schengen: Netlist to Schematic Conversion via Constraint Generation, 2026. Accessed: Apr. 22, 2026. [Online]. Available: <https://anonymous.4open.science/r/schengen/>
- [29] M. Eick and H. E. Graeb, "Mars: Matching-driven analog sizing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 31, no. 8, pp. 1145–1158, 2012.